DF 2026 가을 · 3주차 프로젝트 ① 분류

# 버섯 식용/독성 분류
## 과제 5~7단계 해설

교차검증 → GridSearchCV → 파라미터 선택 이유까지, 코드 한 줄씩과 결과 해석, 캔바 PPT 문구를 한 번에 정리했어요.

| 99.97% | 100% | 12 → 1 | depth 5 |
|---|---|---|---|
| 결정 트리 교차검증 평균 (Stratified 5-fold) | 랜덤 포레스트 교차검증 평균 (5번 모두 만점) | 탐색 조합 12개 중 최종 1개 (동점 6개) | 결정 트리 최종 설정 (질문 최대 5번) |

### 전체 흐름

**0. 이어서 준비** (1~4단계 셀 먼저 실행) › **5. 교차검증** (시험을 5번 보고 평균 내기) › **6. GridSearchCV** (설정값 후보를 자동으로 비교) › **6. 점수 비교표** (기본 → 교차검증 → 튜닝 후) › **7. 선택 이유** (근거 정리 PPT에 작성)

### 함께 받은 파일

| 파일 | 용도 |
|---|---|
| mushroom_steps_1_7.ipynb | 1~7단계 전체 노트북. **[13]~[20]이 이번에 새로 추가된 5~7단계**예요. 실행 결과도 들어 있어요. |
| figures.zip | 그래프 PNG( `cv_scores.png` , `depth_cv.png` 등)와 점수 비교표 `score_table.csv` . 캔바에 바로 넣을 수 있어요. |
| mushrooms.csv | 원본 데이터 (지난번에 받은 파일 그대로) |

### 코랩에서 실행하는 두 가지 방법

| 방법 | 하는 법 | 추천 상황 |
|---|---|---|
| A. 덩어리별 실행 | 노트북을 열고 `[0]` 부터 `[20]` 까지 셀을 하나씩 Shift+Enter | 처음 공부할 때 ✅ |
| B. 한 번에 실행 | 메뉴 **런타임 → 모두 실행** | 결과만 빠르게 볼 때 |

⚠️ **꼭 기억하기**

`[13]` 부터는 `[0]~[12]` (1~4단계)를 **먼저 실행해야** 해요. 앞 단계에서 만든 `X_train` , `y_train` , `dt` , `rf` 를 이어서 쓰기 때문이에요. 순서를 건너뛰면 NameError가 나요.

# 먼저, 처음 보는 용어 한눈에

| 용어 | 쉬운 뜻 | 비유 |
|---|---|---|
| 교차검증 (Cross Validation) | 학습 데이터를 여러 조각으로 나눠, 돌아가며 시험 보고 평균 내는 방법 | 모의고사를 한 번이 아니라 5번 보기 |
| 폴드 (fold) | 교차검증에서 나눈 조각 하나. 5-fold = 5조각 | 문제집을 5등분한 한 묶음 |
| Stratified (층화) | 조각마다 식용:독성 비율을 똑같이 맞추는 것 | 모둠을 나눌 때 남녀 비율을 똑같이 |
| 하이퍼파라미터 | 학습 **전에 사람이** 정해주는 설정값 (예: `max_depth`). 모델이 스스로 배우는 값이 아님 | 라면 끓일 때 물 양·시간 |
| GridSearchCV | 후보 설정을 바둑판처럼 늘어놓고 전부 시험해서 가장 좋은 조합을 찾아 주는 도구 | 물 양·시간 조합을 다 끓여 보고 제일 맛있는 걸 고르기 |
| 재현율 (recall) | 실제 독버섯 중에서 "독성"이라고 찾아낸 비율. 독버섯을 놓치지 않는 정도 | 범인 10명 중 몇 명을 잡았나 |
| 표준편차 | 5번 시험 점수가 흔들린 정도. 0에 가까울수록 안정적 | 점수가 들쭉날쭉한 정도 |
| 과적합 | 공부한 문제만 잘 맞히고 새 문제는 못 맞히는 상태 | 기출만 외운 학생 |

**지금까지 어디까지 했나요?**

**1~4단계**: 글자를 숫자로 바꾸고(인코딩) → 문제(X)와 정답(y)을 나누고 → 학습 80%·테스트 20%로 분리 → 결정 트리를 학습시켰더니 테스트 정확도 **1.000**이었어요.

**5~7단계**는 "그 100점이 진짜인지 확인(5) → 모델 설정을 가장 좋은 걸로 고르기(6) → 왜 그렇게 골랐는지 근거 만들기(7)"예요.

# 5단계. 교차검증 (KFold · Stratified KFold)

4단계의 100점은 **시험지 한 장(테스트 1,625개)**으로 잰 점수예요. 그날 시험이 우연히 쉬웠을 수도 있잖아요? 그래서 **시험을 5번** 보고 평균을 내요. 이게 교차검증이에요.

## 교차검증이 돌아가는 모습 (5-fold)




학습 데이터 6,499개를 5조각으로 나눠요. 한 번에 한 조각(주황)만 시험 문제로 쓰고, 나머지 4조각(파랑)으로 공부해요. 시험 볼 조각을 바꿔 가며 5번 하고 점수 5개의 평균을 내요.

| 방식 | 나누는 법 | 특징 |
|---|---|---|
| KFold | 섞은 뒤 5등분 | 조각마다 식용:독성 비율이 조금씩 다를 수 있음 |
| Stratified KFold ✅ | 5등분하되 **조각마다 식용:독성 비율을 똑같이** 유지 | 분류 문제에 기본으로 추천. 6단계에서도 이걸 사용 |

> ⚠️ **교차검증에는 학습 데이터만 써요**
>
> `X_train`, `y_train`만 쓰고 **테스트 데이터는 건드리지 않아요.** 테스트는 마지막 채점용 시험지예요. 중간에 미리 보면 정답을 알고 푸는 셈이라 점수가 진짜가 아니게 돼요.

## 5단계-① KFold vs Stratified KFold 점수

**[13] 5단계-①: KFold vs Stratified KFold 점수 (결정 트리 · 랜덤 포레스트)**

```
from sklearn.model_selection import KFold, StratifiedKFold, cross_val_score

kf  = KFold(n_splits=5, shuffle=True, random_state=42)             # 그냥 5등분
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)   # 클래스 비율을 지키며 5등분

models = {'결정 트리': DecisionTreeClassifier(random_state=42),
          '랜덤 포레스트': RandomForestClassifier(random_state=42)}

fold_scores, cv_rows = {}, []
for name, model in models.items():
    for cv_name, cv in [('KFold', kf), ('StratifiedKFold', skf)]:
        s = cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')   # 5번 학습·채점 → 점수 5개
        fold_scores[(name, cv_name)] = s
        cv_rows.append({'모델': name, '분할 방식': cv_name,
                        '평균': round(s.mean(), 4), '표준편차': round(s.std(), 4),
                        '폴드별 점수': s.round(4).tolist()})
cv_table = pd.DataFrame(cv_rows)
print(cv_table.to_string(index=False))
```

**실행 결과**

| 모델 | 분할 방식 | 평균 | 표준편차 | 폴드별 점수 (5번) |
|---|---|---|---|---|
| 결정 트리 | KFold | 0.9998 | 0.0003 | 1.0, 1.0, 1.0, 0.9992, 1.0 |
| 결정 트리 | StratifiedKFold | 0.9997 | 0.0006 | 0.9985, 1.0, 1.0, 1.0, 1.0 |
| 랜덤 포레스트 | KFold | 1.0000 | 0.0000 | 1.0, 1.0, 1.0, 1.0, 1.0 |
| 랜덤 포레스트 | StratifiedKFold | 1.0000 | 0.0000 | 1.0, 1.0, 1.0, 1.0, 1.0 |

### 코드 한 줄씩

| 코드 | 뜻 |
|---|---|
| `KFold(n_splits=5, shuffle=True, random_state=42)` | 5조각으로 나누는 도구. `shuffle=True`는 섞고 나누기, `random_state=42`는 누가 해도 같은 조각이 나오게 고정 |
| `StratifiedKFold(...)` | 같은 5조각이되 조각마다 식용:독성 비율을 유지하는 버전 |

| 코드 | 뜻 |
|---|---|
| `cross_val_score(model, X_train, y_train, cv=..., scoring='accuracy')` | 5번 학습·채점해서 **점수 5개**를 돌려줌 (`cv`에 나누는 방식을 넣음) |
| `s.mean() / s.std()` | 평균 / 표준편차. 표준편차가 0에 가까울수록 점수가 안정적 |

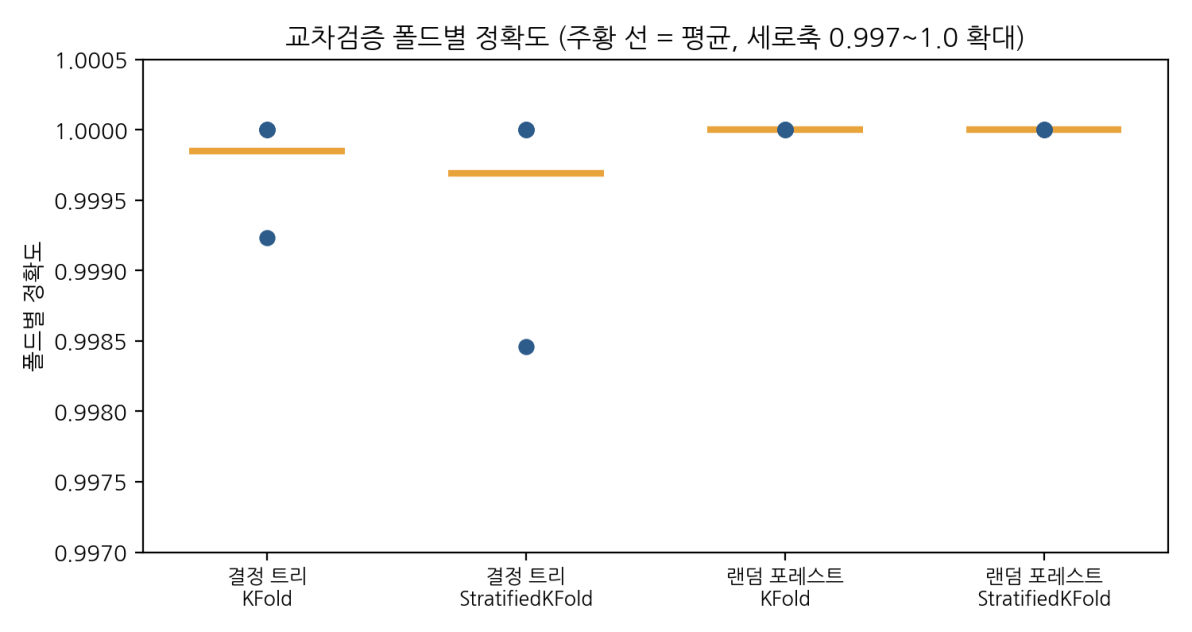
### 결과 읽는 법

- **랜덤 포레스트**는 5번 모두 100%였어요.
- **결정 트리**는 5번 중 4번이 100%이고, 딱 한 번만 1~2개를 틀렸어요 (Stratified의 첫 조각 0.9985 = 약 1,300개 중 2개 오답).
- 그래서 4단계의 100점은 우연히 쉬운 시험지 덕분이 아니었어요. 어느 조각으로 시험 봐도 거의 만점이에요.
- 두 방식의 점수가 거의 같은 건, 이 데이터가 식용 51.8% : 독성 48.2%로 원래 균형이 맞아서예요. 그래도 안전하게 Stratified를 기본으로 써요.

## 5단계-② 폴드별 점수 그래프

**[14] 5단계-②: 폴드별 점수 그래프**

```
fig, ax = plt.subplots(figsize=(7.5, 4))
labels = []
for i, ((name, cv_name), s) in enumerate(fold_scores.items()):
    ax.scatter([i] * len(s), s, color='#2E5C8A', s=40, zorder=3)              # 파란 점 = 폴드 하나의 점수
    ax.hlines(s.mean(), i - 0.3, i + 0.3, color='#E8A33B', lw=3, zorder=2)    # 주황 선 = 평균
    labels.append(f'{name}\n{cv_name}')
ax.set_xticks(range(len(labels))); ax.set_xticklabels(labels, fontsize=9)
ax.set_ylim(0.997, 1.0005); ax.set_ylabel('폴드별 정확도')
ax.set_title('교차검증 폴드별 정확도 (주황 선 = 평균, 세로축 0.997~1.0 확대)')
plt.tight_layout(); plt.savefig('figures/cv_scores.png', dpi=200); plt.show()
```




- **파란 점** = 시험 한 번의 점수, **주황 선** = 5번의 평균이에요. 랜덤 포레스트는 점 5개가 모두 1.0에 겹쳐 있어요.
- 세로축을 **0.997~1.0만 확대**해서 작은 차이가 커 보여요. 실제 차이는 0.2%p도 안 돼요. 발표할 때는 이 점을 말해 주세요.

# 6단계. GridSearchCV로 최적 파라미터 탐색

4단계에서 `max_depth`(질문 최대 횟수)를 바꾸면 성능이 달라지는 걸 봤어요. 어떤 값이 제일 좋은지 하나씩 직접 시험하는 대신, **GridSearchCV**가 **후보 조합을 전부 시험**해서 가장 좋은 조합을 찾아 줘요. 이런 "사람이 정해주는 설정값"을 **하이퍼파라미터**라고 해요.

**GridSearchCV가 하는 일**

- ① 후보 설정을 바둑판(grid)처럼 늘어놓아요.
- ② 조합마다 5단계의 교차검증(Stratified 5-fold)으로 점수를 매겨요.
- ③ 점수가 가장 높은 조합을 알려 줘요. 그 조합이 `best_params_`, 점수가 `best_score_` 예요.

그래서 **정확도 비교와 최적 파라미터 탐색을 한 번에** 해 줘요.

## 무엇을 후보로 넣었나요?

| 설정 이름 | 뜻 | 후보 | 개수 |
|---|---|---|---|
| `max_depth` | 질문 최대 횟수 (None = 제한 없음) | 2, 3, 4, 5, 6, None | 6 |
| `criterion` | 질문할 때 칸을 나누는 기준 | gini, entropy | 2 |

→ 6 × 2 = **12개 조합** × 5-fold = 60번 학습해서 비교해요.

## 6단계-① GridSearchCV로 결정 트리 최적 파라미터 탐색

**[15] 6단계-①: GridSearchCV로 결정 트리 최적 파라미터 탐색**

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'max_depth': [2, 3, 4, 5, 6, None],    # 질문 최대 횟수 (None = 제한 없음)
    'criterion': ['gini', 'entropy'],      # 칸을 나누는 기준
}

gs = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid,
                  cv=skf, scoring='accuracy')     # skf: [13]에서 만든 Stratified 5-fold
gs.fit(X_train, y_train)                          # 학습 데이터로만 탐색

print('선택된 설정      :', gs.best_params_)
print('교차검증 정확도  :', round(gs.best_score_, 4))
```

**실행 결과**

```
선택된 설정      : {'criterion': 'gini', 'max_depth': 5}
교차검증 정확도  : 0.9997
```

### 코드 한 줄씩

| 코드 | 뜻 |
|---|---|
| `param_grid = {...}` | 시험해 볼 후보 설정을 적은 표 |

| 코드 | 뜻 |
|---|---|
| `GridSearchCV(모델, param_grid, cv=skf, scoring='accuracy')` | 모든 조합을 Stratified 5-fold로 시험하고 **정확도**로 점수를 매김 ( `skf` 는 [13]에서 만든 것) |
| `gs.fit(X_train, y_train)` | **학습 데이터로만** 탐색. 테스트는 사용하지 않음 |
| `gs.best_params_` | 가장 점수 높은 설정 |
| `gs.best_score_` | 그 설정의 교차검증 평균 정확도 |
| `gs.best_estimator_` | 그 설정으로 학습까지 끝난 최종 모델 ([17]에서 사용) |

### 결과 읽는 법

- 12개 조합 중 **가장 좋은 설정은** `gini · max_depth=5` , 교차검증 정확도 99.97%예요.
- 사실 점수가 같은 조합이 6개 있어요(아래 [16]). 동점이면 목록에서 먼저 나온 조합이 선택되는데, 후보를 얕은 깊이부터 적어 둬서 **가장 단순한 깊이 5**가 뽑혔어요.

## 6단계-② 조합별 교차검증 점수

**[16] 6단계-②: 조합별 교차검증 점수 (위에서부터 점수 높은 순)**

```
cv_df = pd.DataFrame(gs.cv_results_)
cols = ['param_max_depth', 'param_criterion', 'mean_test_score', 'rank_test_score']
print(cv_df[cols].sort_values('rank_test_score').head(8).round(4).to_string(index=False))
```

실행 결과

| max_depth | criterion | 교차검증 정확도 | 순위 |
|---|---|---|---|
| 5 | gini | 0.9997 | 1 |
| 6 | gini | 0.9997 | 1 |
| None | gini | 0.9997 | 1 |
| 5 | entropy | 0.9997 | 1 |
| 6 | entropy | 0.9997 | 1 |
| None | entropy | 0.9997 | 1 |
| 4 | gini | 0.9926 | 7 |
| 4 | entropy | 0.9883 | 8 |

- 깊이 **5 · 6 · 제한 없음**은 gini든 entropy든 모두 99.97%로 **동점(순위 1)**이에요.
- 깊이 4는 99.26%(gini), 98.83%(entropy)로 뚝 떨어져요. 그래서 **5가 "충분한 가장 얕은 깊이"**예요.

## 6단계-③ 점수 비교표 (기본 → 교차검증 평균 → 튜닝 후)

팀 PPT의 **점수 비교표(분류)**를 채우는 셀이에요. 테스트 점수는 **마지막에 딱 한 번**만 재요.

**[17] 6단계-③: 점수 비교표 (기본 → 교차검증 평균 → 튜닝 후) 팀 PPT의 **점수 비교표(분류)** 를 채우는 셀이에요. 테스트 점수는 **마지막에 딱 한 번** 재요.**

```
best_dt = gs.best_estimator_                    # 최적 설정으로 학습이 끝난 모델
base_pred = dt.predict(X_test)                  # 4단계 기본 모델 (튜닝 전)
tuned_pred = best_dt.predict(X_test)            # 튜닝 후 모델

score_table = pd.DataFrame({
    '단계': ['① 기본 모델 (테스트)', '② 교차검증 평균 (Stratified 5)', '③ 튜닝 후 (교차검증)', '③ 튜닝 후 (테스트)'],
    '정확도': [accuracy_score(y_test, base_pred),
              fold_scores[('결정 트리', 'StratifiedKFold')].mean(),
              gs.best_score_,
              accuracy_score(y_test, tuned_pred)],
})
print(score_table.round(4).to_string(index=False))
print('놓친 독버섯 (기본 → 튜닝 후):', confusion_matrix(y_test, base_pred)[1, 0], '→', confusion_matrix(y_test,
tuned_pred)[1, 0])
score_table.to_csv('figures/score_table.csv', index=False, encoding='utf-8-sig')
```

**실행 결과**

| 단계 | 정확도 |
|---|---|
| ① 기본 모델 (테스트) | 1.0000 |
| ② 교차검증 평균 (Stratified 5) | 0.9997 |
| ③ 튜닝 후 (교차검증) | 0.9997 |
| ③ 튜닝 후 (테스트) | 0.9988 |

```
놓친 독버섯 (기본 → 튜닝 후): 0 → 2
```

**❓ 튜닝했는데 테스트 점수가 더 낮아졌어요 (100% → 99.88%)**

- 테스트 1,625개 중 **2개 차이**예요. 한 번 나눈 시험지에서 생긴 오차 수준이에요.
- **교차검증 점수는 99.97%로 똑같아요.** 기본 모델(깊이 제한 없음)과 깊이 5가 동점이라는 뜻이에요.
- "테스트 점수가 더 좋은 쪽"을 고르면 시험지를 보고 답을 고르는 셈이라 **하지 않았어요.** 고르는 건 교차검증 점수로만 했어요.

# 7단계. 파라미터 선택 이유 (PPT에 쓸 근거 만들기)

발표에서 "왜 `max_depth`를 5로 했나요?"라는 질문이 꼭 나와요. 그때 **숫자와 그래프로 답할 수 있게** 근거를 모으는 단계예요. 중요한 건 **테스트 점수가 아니라 교차검증 점수**로 골랐다는 점이에요.

## 7단계-① max_depth별 교차검증 점수

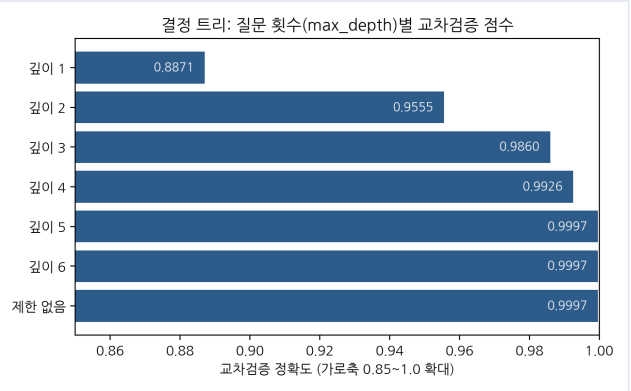
**[18] 7단계-①: max_depth별 교차검증 점수 그래프**

```
depths = [1, 2, 3, 4, 5, 6, None]
cv_depth = [cross_val_score(DecisionTreeClassifier(max_depth=d, random_state=42),
                            X_train, y_train, cv=skf, scoring='accuracy').mean() for d in depths]
names = ['제한 없음' if d is None else f'깊이 {d}' for d in depths]
print(pd.DataFrame({'max_depth': names, '교차검증 정확도': np.round(cv_depth, 4)}).to_string(index=False))

fig, ax = plt.subplots(figsize=(6.5, 4))
ax.barh(names[::-1], cv_depth[::-1], color='#2E5C8A')
for i, v in enumerate(cv_depth[::-1]):
    ax.text(v - 0.003, i, f'{v:.4f}', va='center', ha='right', color='white', fontsize=9)
ax.set_xlim(0.85, 1.0); ax.set_xlabel('교차검증 정확도 (가로축 0.85~1.0 확대)')
ax.set_title('결정 트리: 질문 횟수(max_depth)별 교차검증 점수')
plt.tight_layout(); plt.savefig('figures/depth_cv.png', dpi=200); plt.show()
```

**실행 결과**

| max_depth | 교차검증 정확도 |
|---|---|
| 깊이 1 | 0.8871 |
| 깊이 2 | 0.9555 |
| 깊이 3 | 0.9860 |
| 깊이 4 | 0.9926 |
| 깊이 5 | **0.9997** |
| 깊이 6 | 0.9997 |
| 제한 없음 | 0.9997 |




- 질문을 **1번**만 해도(냄새만 봐도) 88.7%예요.
- 질문을 늘릴수록 오르다가 **5번에서 99.97%**에 닿고, 그 뒤로는 **더 안 올라가요.**
- → "더 깊게 키울 이유가 없어서 **5**로 정했다"고 말하면 돼요.

## 7단계-② 설정값별 최고 점수

**[19] 7단계-②: 설정값별 최고 점수 (어떤 값이 점수를 바꿨나?)**

```
for p in ['param_max_depth', 'param_criterion']:
    key = cv_df[p].apply(lambda v: 'None' if v is None else str(v))          # None도 글자로 바꿔서 함께 집계
    g = cv_df.groupby(key)['mean_test_score'].max().round(4)                 # 그 값을 쓴 조합 중 최고 점수
    print(f"{p.replace('param_', ''):10s}", ' | '.join(f'{k}: {v}' for k, v in g.items()))
```

**실행 결과**

```
max_depth  2: 0.9555 | 3: 0.986 | 4: 0.9926 | 5: 0.9997 | 6: 0.9997 | None: 0.9997
criterion  entropy: 0.9997 | gini: 0.9997
```

- 한 줄은 "그 설정값을 쓴 조합들 중 **최고 점수**"예요.
- 값을 바꾸면 점수가 변하는 설정(`max_depth`)은 **중요한 설정**이에요.
- 값을 바꿔도 점수가 같은 설정(`criterion`)은 **어느 걸 써도 상관없다**는 뜻이라, 더 단순하고 빠른 쪽을 골라요.

## 7단계-③ 선택 이유 정리 (PPT에 그대로 옮기기)

| 파라미터 | 후보 | 선택 | 선택 이유 |
|---|---|---|---|
| `max_depth` | 2·3·4·5·6·None | **5** | 깊이 1~4는 점수가 계속 오르지만(88.7%→99.3%) **5부터 99.97%에서 멈춤**. 더 깊게 키울 이유가 없고, 5·6·제한 없음이 동점이라 가장 단순한 5를 선택 |
| `criterion` | gini · entropy | **gini** | 두 기준의 최고 점수가 같아서(99.97%) 계산이 더 빠른 gini |

**선택 기준 (발표에서 이대로 말하면 돼요)**

① **교차검증 정확도가 가장 높은 조합 → ② 동점이면 더 단순한 설정.** 테스트 점수는 고르는 데 쓰지 않았어요.

**이 데이터에서 튜닝의 의미**

이 데이터는 규칙이 뚜렷해서 점수가 거의 천장(100%)이에요. 그래서 튜닝의 의미는 점수를 올리는 게 아니라 **"깊이 5면 충분하다"는 근거**를 얻는 거예요.

# 캔바 PPT에 옮길 내용

조별 PPT 구성의 **교차검증 · 튜닝 · 파라미터 선택 이유 · 점수 비교표** 부분이에요. 아래 문구를 그대로 옮기고, 그림은 `figures` 폴더의 PNG를 쓰면 돼요.

## 슬라이드 C. 교차검증

- 학습 데이터 6,499개를 5조각으로 나눠 5번 평가 (테스트 데이터는 사용하지 않음)
- KFold: 결정 트리 99.98%, 랜덤 포레스트 100%
- Stratified KFold: 결정 트리 99.97%, 랜덤 포레스트 100% (조각마다 식용:독성 비율 유지)
- 결정 트리는 5번 중 1번만 1~2개 틀림 → 4단계의 100%가 우연이 아님을 확인
- 📸 그림 추천: `cv_scores.png`

## 슬라이드 D. GridSearchCV

- 결정 트리의 max_depth 6가지 × criterion 2가지 = 12조합을 Stratified 5-fold로 비교
- 선택 결과: `criterion=gini`, `max_depth=5` (교차검증 정확도 0.9997)
- 깊이 5·6·제한 없음이 99.97%로 동점 → 가장 단순한 5 선택, 깊이 4는 99.26%로 하락

## 슬라이드 E. 파라미터 선택 이유

- `max_depth=5` : 깊이 1~4는 88.7% → 99.3%로 상승, 5부터 99.97%에서 정체 → 더 깊게 할 이유 없음
- `criterion=gini` : gini·entropy 최고 점수 동일 → 계산이 빠른 gini
- 선택 기준: 교차검증 정확도 → 동점이면 단순한 설정 (테스트 점수는 선택에 사용하지 않음)
- 📸 그림 추천: `depth_cv.png`

## 점수 비교표 (분류)

| 모델 | 기본 모델 (테스트) | 교차 검증 평균 | 튜닝 후 (교차검증) | 튜닝 후 (테스트) |
|---|---|---|---|---|
| 결정 트리 | 1.000 | 0.9997 | 0.9997 | 0.9988 |

표 아래 한 줄 해석: 튜닝 후 테스트 0.9988은 1,625개 중 2개 차이이고 교차검증 점수는 같음 → 사실상 동일 성능. 팀 PPT 1번 슬라이드의 **(e : 1, p : 0)**은 **(e : 0, p : 1)**로 고쳐 주세요.

# 예상 질문 & 답

| 질문 | 답 |
| --- | --- |
| 교차검증은 왜 하나요? | 테스트 한 번의 점수는 시험지가 쉬워서 높을 수도 있어서요. 학습 데이터를 5번 나눠 시험을 반복했고, 결과가 99.97~100%로 안정적이었어요. |
| KFold와 Stratified KFold는 뭐가 다른가요? | KFold는 섞어서 5등분하고, Stratified는 조각마다 식용:독성 비율을 똑같이 맞춰요. 이 데이터는 원래 균형(51.8:48.2)이라 점수 차이는 거의 없었지만 분류에서는 Stratified를 기본으로 써요. |
| 교차검증에 테스트 데이터를 왜 안 썼나요? | 테스트 데이터는 마지막 채점용 시험지예요. 설정을 고르는 데 미리 보면 정답을 알고 푸는 셈이라 점수가 진짜가 아니게 돼요. |
| GridSearchCV가 뭔가요? | 후보 설정을 전부 조합해서 교차검증으로 점수를 매기고, 가장 좋은 조합을 자동으로 찾아 주는 도구예요. 이번에는 12개 조합을 5-fold로 비교했어요. |
| 튜닝했는데 테스트 점수가 더 낮은데요? | 테스트 1,625개 중 2개 차이이고, 교차검증 점수는 99.97%로 똑같아요. 깊이 5·6·제한 없음이 동점이라 가장 단순한 5를 골랐어요. 테스트 점수를 보고 고르면 시험지를 보고 답을 고르는 셈이라 하지 않았어요. |
| 왜 max_depth=5인가요? | 깊이 1~4는 점수가 계속 올랐지만(88.7→99.3%) 5부터 99.97%에서 멈췄어요. 더 깊게 해도 오르지 않아서 가장 단순한 5를 골랐어요. |
| 정확도 100%면 튜닝이 필요 없지 않나요? | 맞아요, 이 데이터는 규칙이 뚜렷해서 점수가 거의 천장이에요. 그래서 튜닝의 의미는 점수를 올리는 게 아니라 "깊이 5면 충분하다"는 근거를 얻고 가장 단순한 설정을 고르는 거예요. |
| 랜덤 포레스트는 왜 튜닝하지 않았나요? | 팀에서 메인 모델을 결정 트리로 정했어요. 랜덤 포레스트는 기본 설정만으로 교차검증 5번 모두 100%였고, 튜닝 없이도 비교용으로 충분했어요. |

# 자주 나는 에러

| 에러 | 원인 | 해결 |
|---|---|---|
| NameError: name 'X_train' is not defined<br>(또는 'dt', 'rf') | 1~4단계 셀을 건너뛰고 [13]부터 실행 | `[0]` 부터 순서대로 실행, 또는 런타임 → 모두 실행 |
| NameError: name 'skf' is not defined | [13]을 건너뛰고 [15]나 [18]을 실행 | `[13]` 을 먼저 실행 |
| NameError: name 'gs' (또는 'cv_df') is not defined | [15]·[16]을 건너뛰고 [17]이나 [19]를 실행 | `[15]`, `[16]` 을 먼저 실행 |
| 결과 숫자가 이 문서와 다름 | `random_state` 를 빼거나 바꿈 | `random_state=42` 그대로 두기 |
| 그래프 한글이 □□ | 글꼴 미설정 | `[0]` 덩어리 실행 |
| 에러가 줄줄이 수십 개 | 앞 덩어리 하나가 실패해서 뒤가 따라 실패 | 첫 번째 에러만 해결 → 런타임 → 세션 다시 시작 → 모두 실행 |

# 다음 할 일

- ✅ 5단계 교차 검증 (KFold, Stratified KFold) → 폴드별 점수와 평균
- ✅ 6단계 GridSearchCV → 최적 파라미터, 튜닝 전후 점수
- ✅ 7단계 파라미터 선택 이유 정리 → 위 슬라이드 C·D·E 문구
- ☐ 캔바 PPT에 슬라이드 C·D·E와 점수 비교표를 넣고, `cv_scores.png`, `depth_cv.png` 붙이기
- ☐ 1번 슬라이드의 (e : 1, p : 0)을 (e : 0, p : 1)로 고치기
- ☐ 노션 A반 과제 3주차에 업로드